Manual Técnico
Módulo Presupuestos

Versión 2.0 | 24/08/2026

Documentación técnica para desarrolladores y administradores del sistema.

Índice de Contenido

#	Sección
1.	Arquitectura del Sistema
2.	Estructura de Archivos
3.	Base de Datos
3.1	Tabla presupuestos
3.2	Tabla partidas
3.3	Tabla items_partida
3.4	Tabla partida_paquetes
3.5	Tabla parametros_sistema
3.6	Relaciones
4.	Dependencias
5.	Módulos y Clases Principales
5.1	Clase PresupuestosWindow
5.2	Clase Service
5.3	Clase PresupuestoController
5.4	Clase PaquetesController
5.5	Clase FocusManager
5.6	Calculadoras (calculators/)
5.7	Generador PDF
6.	Flujo de Datos
6.1	Creación de Presupuesto
6.2	Agregar Material
6.3	Agregar Paquete
6.4	Guardado de Partida
6.5	Impresión PDF
7.	Manejo de Errores

8.	Logging y Depuración
9.	Configuración
10.	Integración con otros Módulos

1. Arquitectura del Sistema

El Módulo de Presupuestos sigue una arquitectura de tres capas:

Capa	Tecnología	Responsabilidad
Presentación	PyQt6	Interfaz gráfica (PresupuestosWindow)
Controladores	Python 3.12	PresupuestoController, PaquetesController, RecalculoGlobalController
Lógica de Negocio	Python + Calculators	Dispatch de cálculos por tipo
Acceso a Datos	pyodbc	Clase Service con conexión a múltiples BD
Almacenamiento	SQL Server	Bases de datos: presupuestos, clientes, aluminio, herrajes, herreria, plasticos, otı

[Imagen pendiente: arquitectura_sistema.png]

2. Estructura de Archivos

Archivo	Descripción
modulo_presupuestos.py	UI principal (QMainWindow) con toda la interfaz y menú
service.py	Capa de datos, conexión a BD, CRUD, precios (Fachada)
busquedas.py	Búsqueda de clientes y materiales con autocompletado
foco.py	FocusManager - navegación por teclado centralizada
rules_zero_price.py	Validación de precios > 0 antes de agregar
calculators/	Calculadoras específicas por tipo de material
controllers/	Controladores de presupuesto, paquetes y recálculo global
core/	Constantes y utilidades
ui/	Diálogos y tablas especializadas
reports/	Generación de PDF y corrector ortográfico

[Imagen pendiente: estructura_modulos.png]

3. Base de Datos

3.1 Tabla presupuestos (presupuestos.dbo.presupuestos)

Columna	Tipo	Nulo	Descripción
id	int	NO	PK - ID autoincremental
cliente_id	int	NO	FK → clientes.dbo.Clientes.id
obra	varchar(50)	NO	Nombre de la obra
presupuesto	int	NO	Número de presupuesto (único)
fecha	date	NO	Fecha del presupuesto
color_global	varchar(30)	NO	Color global o 'VARIOS'
descuento_porcentaje	decimal(5,2)	YES	Descuento global
notas	text	YES	Notas aclaratorias
forma_pago	varchar(150)	YES	Forma de pago
plazo_entrega	varchar(150)	YES	Plazo de entrega

3.2 Tabla partidas (presupuestos.dbo.partidas)

Columna	Tipo	Descripción
id	int	PK autoincremental
presupuesto_id	int	FK → presupuestos.id
partida	int	Número de partida (1,2,3...)
titulo	varchar(100)	Título de la partida
descripcion	text	Descripción detallada
color_partida	varchar(30)	Color específico o vacío
desperdicio	decimal(5,2)	Porcentaje de desperdicio
fv_porcentaje	decimal(5,2)	Factor de venta %
kg	decimal(10,2)	Kilos totales (herrería)
ancho	decimal(10,2)	Ancho global de la partida
alto	decimal(10,2)	Alto global de la partida
mano_obra	decimal(10,2)	Monto de mano de obra

accesorios	decimal(10,2)	Monto de accesorios
piezas	int	Número de piezas
precio_unitario	decimal(12,2)	P.U. de la partida
total_partida	decimal(12,2)	Importe total de la partida

3.3 Tabla items_partida (presupuestos.dbo.items_partida)

Columna	Tipo	Descripción
id	int	PK autoincremental
partida_id	int	FK → partidas.id
tipo	varchar(30)	AL/HR/HE/VI/PL/OT/PH
color	varchar(30)	Color del material
clave	varchar(50)	Clave del material
descripcion	text	Descripción
ca	decimal(10,2)	Cantidad Ancho (tramos)
ancho	decimal(10,2)	Ancho en metros
ch	decimal(10,2)	Cantidad Alto (tramos)
alto	decimal(10,2)	Alto en metros
piezas	int	Número de piezas
pu	decimal(12,2)	Precio Unitario
importe	decimal(12,2)	Importe total
metros	decimal(10,2)	Metros lineales (herrería)
kg_m	decimal(10,3)	Kg/m (herrería)
kgs	decimal(10,3)	Kilos totales (herrería)
precio_kg	decimal(12,2)	Precio por kg (herrería)

3.4 Tabla partida_paquetes (presupuestos.dbo.partida_paquetes)

Columna	Tipo	Descripción
id	int	PK autoincremental
partida_id	int	FK → partidas.id

tipo_paquete	varchar(50)	PAQUETE ALUMINIO / PAQUETE HERRAJES
clave_paquete	varchar(50)	Clave del paquete
descripcion	varchar(255)	Descripción del paquete
ancho	decimal(10,2)	Ancho del paquete
alto	decimal(10,2)	Alto del paquete
piezas	int	Piezas del paquete

3.5 Tabla parametros_sistema (presupuestos.dbo.parametros_sistema)

Columna	Tipo	Descripción
id	int	PK (siempre 1)
correo_remitente	varchar(100)	Correo para envíos
contrasena_correo	varchar(100)	Contraseña del correo
servidor_smtp	varchar(100)	Servidor SMTP
puerto_smtp	int	Puerto SMTP
numero_inicial_presupuesto	int	Número inicial para nuevos presupuestos
iva_porcentaje	decimal(5,2)	Porcentaje de IVA
clave_acceso_mantenimiento	varchar(50)	Clave de administrador
color_global_defecto	nvarchar(30)	Color global por defecto
desperdicio_defecto	float	Desperdicio por defecto (%)
fv_defecto	float	Factor de Venta por defecto (%)

3.6 Relaciones y Claves Foráneas

- FK_items_partida_partidas: items_partida.partida_id → partidas.id (ON DELETE CASCADE)
- FK_partida_paquetes_partida: partida_paquetes.partida_id → partidas.id (ON DELETE CASCADE)
- FK_partidas_presupuestos: partidas.presupuesto_id → presupuestos.id
- FK_resumen_presupuesto_presupuesto: resumen_presupuesto.presupuesto_id → presupuestos.id
- presupuestos.cliente_id → clientes.Clientes.id (relación semántica, sin FK explícita)

[Imagen pendiente: diagrama_bd.png]

4. Dependencias

Librería	Versión	Uso
PyQt6	6.5+	Interfaz gráfica completa
pyodbc	4.0+	Conexión ODBC a SQL Server (múltiples BD)
reportlab	3.6+	Generación de PDF profesional
pyspellchecker	0.6+	Corrector ortográfico (opcional)
Python	3.12+	Lenguaje base

```
pip install PyQt6 pyodbc reportlab pyspellchecker
```

5. Módulos y Clases Principales

5.1 Clase PresupuestosWindow (modulo_presupuestos.py)

- _build_ui() - Construye toda la interfaz
- _build_menu() - Construye el menú principal (Archivo, Presupuesto, Mantenimiento, Ayuda)
- _on_nuevo_presupuesto() - Crea nuevo presupuesto
- _on_abrir_presupuesto() - Abre diálogo de búsqueda
- _on_guardar_presupuesto() - Guarda presupuesto
- _on_nueva_partida() - Crea nueva partida
- _on_imprimir() / _on_vista_previa() - Generación PDF
- _on_recalcular_todo() - F5 - Recalcula todas las filas
- _add_std() - Agrega material a tabla estándar
- _add_he() - Agrega material a tabla herrería
- _calc_std_row() / _calc_he_row() - Cálculo por fila
- _recalc_summary() - Actualiza totales
- _sync_color_rule() - Maneja colores globales
- _lanzar_catalogo_unico() - Abre catálogos con instancia única
- eventFilter() - Captura teclas

```
# Atajos de teclado
Ctrl+N → _on_nuevo_presupuesto
Ctrl+O → _on_abrir_presupuesto
Ctrl+S → _on_guardar_presupuesto
Ctrl+Shift+S → _on_guardar_como_presupuesto
Ctrl+P → _on_imprimir
Ctrl+T → _on_nueva_partida
Del → _on_eliminar_partida
F5 → _on_recalcular_todo
F1 → _on_contenido_ayuda
```

5.2 Clase Service (service.py)

- connect() / cursor() - Conexión a SQL Server con autodetección de drivers
- buscar_por_tabla() / buscar_por_tipo() - Búsqueda de materiales
- clientes_buscar() - Búsqueda de clientes por nombre completo
- aluminio_precio(), aluminio_largo() - Precios y largo de aluminio
- vidrio_precio() - Precio de vidrio por m²
- herreria_kg_por_m() - Kg/m de herrería

- herrajes_precio(), plasticos_precio(), otros_precio() - Precios unitarios
- paquete_aluminio_items() - Componentes de paquete de aluminio
- paquete_herrajes_items() - Componentes de paquete de herrajes
- guardar_presupuesto(), guardar_partida(), guardar_items_partida(), guardar_paquetes_partida()
- obtener_presupuesto(), obtener_partidas(), obtener_items_partida(), obtener_paquetes_partida()
- obtener_ultimo_numero_presupuesto() - Para consecutivo automático
- buscar_presupuestos() - Búsqueda avanzada con filtros
- obtener_parametros_sistema(), guardar_parametros_sistema()
- obtener_clave_admin(), cambiar_clave_admin() - Gestión de clave admin
- obtener_configuracion_presupuesto(), guardar_configuracion_presupuesto()

5.3 Clase PresupuestoController (controllers/presupuesto_controller.py)

- guardar_partida_actual_silencioso() - Guardado automático al cambiar partida
- partida_anterior() / partida_siguiente() - Navegación
- cargar_partida_por_indice() - Desde ComboBox
- crear_nueva_partida() - Creación de partida
- duplicar_partida_actual() - Duplicación
- eliminar_partida_actual() - Eliminación
- cargar_presupuesto() - Carga completa desde BD
- recalcular_por_color() - Recalcula al cambiar color
- propagar_desperdicio_fv() - Propaga desperdicio/FV a todas las partidas
- duplicar_presupuesto_actual() - Duplica presupuesto completo
- exportar_csv() - Exporta a CSV
- exportar_pdf() - Genera PDF
- vista_previa_pdf() - Vista previa con QPdfView

5.4 Clase PaquetesController (controllers/paquetes_controller.py)

- _cargar_componentes_base() - Carga desde BD con expansión recursiva de PH
- _expandir_componentes_ph() - Expande paquetes de herrajes anidados
- _consolidar_a_materiales() - Toma componentes y los agrega a la tabla de materiales
- _mostrar_en_anidada() - Muestra componentes en tabla visual (solo informativa)

- _on_piezas_cambiadas() / _on_ancho_cambiado() / _on_alto_cambiado() - Actualiza consolidación
- Fórmula: cantidad_final = cantidad_base × multiplicador_ph × piezas_paquete

5.5 Clase FocusManager (foco.py)

- FOCUS_FLOW - Orden de navegación por campos
- _move_focus_global() - Navegación entre widgets principales
- _move_in_table() - Navegación dentro de tablas
- _handle_list_enter() - Selección en listas de búsqueda
- eventFilter() - Filtro global de teclas
- Soporte para Tab, Shift+Tab, Enter, Escape, Flechas, Delete

5.6 Calculadoras (calculators/)

- dispatch_calculo(ctx) - Router central por tipo de material
- aluminio.compute_aluminio() - Fórmula: (CA × Ancho + CH × Alto) × (Precio_color / (Largo - 0.10))
- herreria.compute_herreria() - Kgs = ((CA×Ancho)+(CH×Alto)) × Kg/m → P.U. = Kgs × $/kg
- vidrio.compute_vidrio() - m² = Ancho × Alto → P.U. = m² × Precio_m2
- paquete_aluminio.compute_paquete_aluminio_inline() - Expande componentes recursivamente
- paquete_herrajes.compute_paquete_herrajes_inline() - Expande herrajes recursivamente
- dimensioning.adjust_dimension() - Redondeo industrial a múltiplo de 0.05

5.7 Generador PDF (reports/)

- PresupuestoPDFService - Servicio principal de generación PDF
- generar_pdf_desde_id() - Genera PDF desde ID de presupuesto
- VistaPreviaPDFDialog - Diálogo de vista previa con QPdfView
- NotasManager - Gestión de notas, forma de pago, plazo
- CorrectorOrtografico - Corrector ortográfico en tiempo real
- TextEditConCorrector - QTextEdit con subrayado de errores

6. Flujo de Datos

6.1 Creación de Presupuesto

- 1. Usuario ingresa Cliente (busca o crea nuevo)
- 2. Usuario ingresa Obra, Fecha, Color Global
- 3. Usuario crea partida (Ctrl+T o botón Siguiente)
- 4. Usuario ingresa Título y Descripción (con corrector ortográfico)
- 5. Usuario configura Desperdicio, F.V., MO, Accesorios
- 6. Usuario agrega materiales (selecciona tipo, busca, selecciona)
- 7. El sistema calcula P.U. e Importe automáticamente
- 8. Usuario repite para más materiales
- 9. Usuario puede agregar paquetes (expansión automática)
- 10. Usuario guarda presupuesto (Ctrl+S o al cambiar partida)

6.2 Agregar Material

- 1. Usuario selecciona Tipo (Aluminio, Herrajes, etc.)
- 2. Usuario selecciona Buscar Por (CLAVE o DESCRIPCIÓN)
- 3. Usuario escribe filtro → textChanged → _search_timer(160ms) → busquedas.do_search()
- 4. service.buscar_por_tipo() → consulta SQL a BD correspondiente
- 5. Resultados en lst_result con UserRole conteniendo los datos
- 6. Usuario selecciona item → on_pick_result()
- 7. Según tipo, llama a _add_std() o _add_he()
- 8. Se inserta fila en tabla y se llama a _calc_std_row()
- 9. dispatch_calculo() → calculadora específica por tipo
- 10. Se actualizan PU, Importe, y _recalc_summary()

6.3 Agregar Paquete

- 1. Usuario selecciona Tipo = PAQUETE ALUMINIO o PAQUETE HERRAJES
- 2. Usuario busca y selecciona paquete → _on_pick_result_paquete()
- 3. tbl_paquetes.agregar_paquete() → dispara paqueteAgregado
- 4. PaquetesController._on_paquete_agregado()

- 5. _cargar_componentes_base() - Carga desde BD (expansión recursiva de PH)
- 6. _mostrar_en_anidada() - Muestra en tabla visual
- 7. _consolidar_a_materiales() - Expande a tabla de materiales
- 8. Si cambia piezas/ancho/alto → re-consolidación automática

6.4 Guardado de Partida

- 1. Usuario cambia de partida o presiona Ctrl+S
- 2. PresupuestoController.guardar_partida_actual_silencioso()
- 3. _recolectar_datos_presupuesto() - Obtiene datos del presupuesto
- 4. _recolectar_datos_partida() - Obtiene datos de partida
- 5. _recolectar_items_partida() - Recorre tablas (tbl, tbl_he)
- 6. _recolectar_paquetes_partida() - Recorre tbl_paquetes
- 7. service.guardar_presupuesto() - INSERT/UPDATE
- 8. service.guardar_partida() - INSERT/UPDATE
- 9. service.guardar_items_partida() - DELETE + INSERT
- 10. service.guardar_paquetes_partida() - DELETE + INSERT
- 11. Se emite señal listaPartidasActualizada para actualizar ComboBox

6.5 Impresión PDF

- 1. Usuario presiona Imprimir (Ctrl+P) o Vista Previa
- 2. PresupuestoPDFService.generar_pdf_desde_id(presupuesto_id)
- 3. service.obtener_presupuesto() - Datos del presupuesto
- 4. service.obtener_partidas() - Lista de partidas
- 5. service.obtener_configuracion_presupuesto() - Notas, forma_pago, plazo
- 6. NotasManager.preguntar_notas() - Diálogo opcional
- 7. Construir modelo DatosPresupuesto con partidas
- 8. GeneradorPresupuestoPDF.generar_pdf()
- 9. _dibujar_presupuesto_completo() - Dibuja página por página
- 10. PartidaManager maneja saltos de página (partidas no se dividen)
- 11. Se guarda PDF en D:\vah\pdfs\{numero}.pdf
- 12. Se abre automáticamente el PDF

7. Manejo de Errores

Nivel	Mecanismo	Ejemplo
Validación UI	Comprobaciones en _agregar_item()	Clave vacía → mensaje
Existencia material	rules_zero_price.py	Pregunta si desea agregar
Clave duplicada	paquete_existe() + QMessageBox.question	Pregunta sobrescritura
Conexión BD	try/except + múltiples drivers/variantes	RuntimeError con detalle
Eliminación	Diálogo de confirmación	Mensaje '¿Seguro?'
Impresión	try/except + MessageManager	Error silencioso o crítico según categoría
Guardado automático	Siempre silencioso	Solo log, no molesta al usuario

```
# MessageManager - centralización de mensajes
MessageManager.info(parent, 'Título', 'Mensaje')
MessageManager.warning(parent, 'Título', 'Advertencia')
MessageManager.critical(parent, 'Título', 'Error')
MessageManager.error_calculo(parent, 'Error de cálculo')
MessageManager.confirmar_dimensiones(parent) -> bool
```

8. Logging y Depuración

- MessageManager con categorías: ERROR_CALCULO (silencioso), CONFIRMACION, INFO, ADVERTENCIA, PLACEHOLDER, DEBUG, CRITICO
- Para activar logs de depuración: MessageManager.enable_all()
- Para silenciar: MessageManager.silence_all()
- Los errores de cálculo se registran en consola pero no molestan al usuario
- Para depuración, ejecutar módulo desde consola:
- python -m presupuestos.modulo_presupuestos
- El módulo imprimir.py tiene manejo de excepciones con traceback
- El corrector ortográfico tiene logs en consola

9. Configuración

Cadena de conexión a SQL Server (service.py):

```
CANDIDATE_SERVERS = [
ENV_SERVER or "",
r"DESKTOP-AE60PIU\\SQLEXPRESS",
r"(local)\\SQLEXPRESS",
r".\\SQLEXPRESS",
r"localhost\\SQLEXPRESS",
r"127.0.0.1,1433",
]

def connect(self):
for driver in ("{ODBC Driver 18 for SQL Server}", "{ODBC Driver 17 for SQL Server}"):
for server in CANDIDATE_SERVERS:
try:
return pyodbc.connect(conn_str)
except: continue
raise RuntimeError("No se pudo conectar")
```

Variables de entorno:

```
VAH_SQLSERVER - Sobrescribe el servidor por defecto
Ejemplo: set VAH_SQLSERVER=DESKTOP-OTRO\\SQLEXPRESS
```

Constantes (core/constants.py):

```
# Índices de columnas
C_TIPO=0, C_COLOR=1, C_CLAVE=2, C_DESC=3, C_CA=4, C_ANCHO=5, C_CH=6, C_ALTO=7, C_PZAS=8,
C_PU=9, C_IMP=10
H_TIPO=0, H_CLAVE=1, H_DESC=2, H_CA=3, H_ANCHO=4, H_CH=5, H_ALTO=6, H_METROS=7, H_KG_M=8,
H_KGS=9, H_PKG=10, H_PZAS=11, H_PU=12, H_IMP=13
P_TIPO=0, P_CLAVE=1, P_DESC=2, P_ANCHO=3, P_ALTO=4, P_PZAS=5, P_PU=6, P_IMP=7
A_TIPO=0, A_COLOR=1, A_CLAVE=2, A_DESC=3, A_CA=4, A_ANCHO=5, A_CH=6, A_ALTO=7, A_PZAS=8,
A_PU=9, A_IMP=10
```

```
# Reglas de editabilidad
EDITABILIDAD = {
"ALUMINIO": dict(CA=True, CH=True, ANCHO=True, ALTO=True, PZAS=True),
"HERRAJES": dict(CA=False, CH=False, ANCHO=False, ALTO=False, PZAS=True),
"HERRERÍA": dict(CA=False, CH=False, ANCHO=False, ALTO=False, PZAS=True),
...
}
```

10. Integración con otros Módulos

Módulo	Tabla/Relación	Descripción
Clientes	clientes.dbo.Clientes	FK presupuestos.cliente_id → clientes.id
Materiales Aluminio	aluminio.dbo.materiales_aluminio	Búsqueda de perfiles y precios por color
Materiales Herrajes	herrajes.dbo.materiales_herraje	Búsqueda de herrajes y precios
Materiales Herrería	herreria.dbo.materiales_herreria	Kg/m para cálculo
Materiales Vidrio	vidrio.dbo.materiales_vidrio	Precio por m²
Materiales Plásticos	plasticos.dbo.materiales_plasticos	Precio unitario
Materiales Otros	otros.dbo.materiales_otros	Precio unitario
Paquetes Aluminio	aluminio.dbo.paquetes_aluminio / paquetes_aluminio_items	Explosión de paquetes
Paquetes Herrajes	herrajes.dbo.paquetes_herrajes / paquetes_herrajes_items	Explosión recursiva de herrajes
Parámetros Sistema	presupuestos.dbo.parametros_sistema	Configuración de correo y números iniciales

— Fin del Manual Técnico —

Documento generado el 24/08/2026 | Versión 2.0

VAH ERP © Todos los derechos reservados